Bogdan **Volsky** | C++ Developer

Moscow – Russia
+7 950 481 88-48 • bvolsky2006@gmail.com • bgclutch • @bgclutch

Education

Moscow Institute of Physics and Technology (MIPT) **Moscow**
B.S. in Applied Mathematics and Physics *2024 – 2028*
Phystech School of Radio Engineering and Computer Technology.
GPA: 7.18 / 10

Key Projects

Matrix Library (C++): Designed a generic Matrix library focusing on memory efficiency and exception safety.
- Achieved **Strong Exception Safety** for assignment and modification operations (commit/rollback semantics).
- Implemented a memory management **RAII class**, comparable to **std::array**.

Self-Balancing BST (AVL) (C++): Implemented a high-performance generic AVL tree container.
- Designed a fully **templated** implementation compatible with custom data types.
- Implemented efficient **range queries** and an analog of **std::iterator**.
- Conducted **performance benchmarks** comparing lookups against **std::set**.

LFU Cache (C++): Implemented a high-performance Least Frequently Used cache policy.
- Achieved **O(log N)** time complexity for all operations using a combination of hash maps and linked lists.
- Compared performance against Belady's optimal algorithm.
- Covered with unit tests (Google Test).

Bitonic Sort (OpenCL/C++): High-performance GPU-accelerated implementation of the Bitonic Sort algorithm.
- Implemented a **sorting strategy** leveraging both Global and Local GPU memory.
- Achieved a **3x speedup** over **std::sort** on 10M element datasets.
- Implemented a custom **RAII-based OpenCL wrapper** to manage context, program compilation, and memory buffers, ensuring exception safety.

Pattern Matching (OpenCL/C++): High-throughput string pattern matching system on GPU.
- Implemented a brute-force GPU kernel and optimized kernel with **SoA** and **Constant Cache**.
- Developed a **tiled processing** approach with cooperative loading into **Local Memory** to mitigate memory wall bottlenecks.
- Applied **branchless programming** technique in kernels to maximize execution throughput.

Relevant Courses

Advanced C++: **Industrial Programming in C++ (MIPT).** Instructor: Konstantin Vladimirov.
Grade: Excellent (9/10).
Deep dive into C++17/20 standards. Topics covered: **Move Semantics, Memory Model, Templates, RAII, Exception Safety**.

C Programming: **Industrial Programming in C (MIPT).** Instructor: I.R. Dedinsky.
Focus on memory management, pointer arithmetic, and low-level optimization.

Algorithms: **Algorithms and Data Structures (VK Education).** [Certificate Link]

Technical Skills

Languages: C++ (17/20), C, Python (Scripts/Tests), SQL, Bash

Core Concepts: OOP, RAII, STL, Memory Management, Data Structures, Algorithms

Tools: Git, CMake, AddressSanitizer, Google Test, Linux

Languages

Russian: Native

English: B2 (Upper-Intermediate)